

第十章 系统级I/O

教 师：邹翔宇

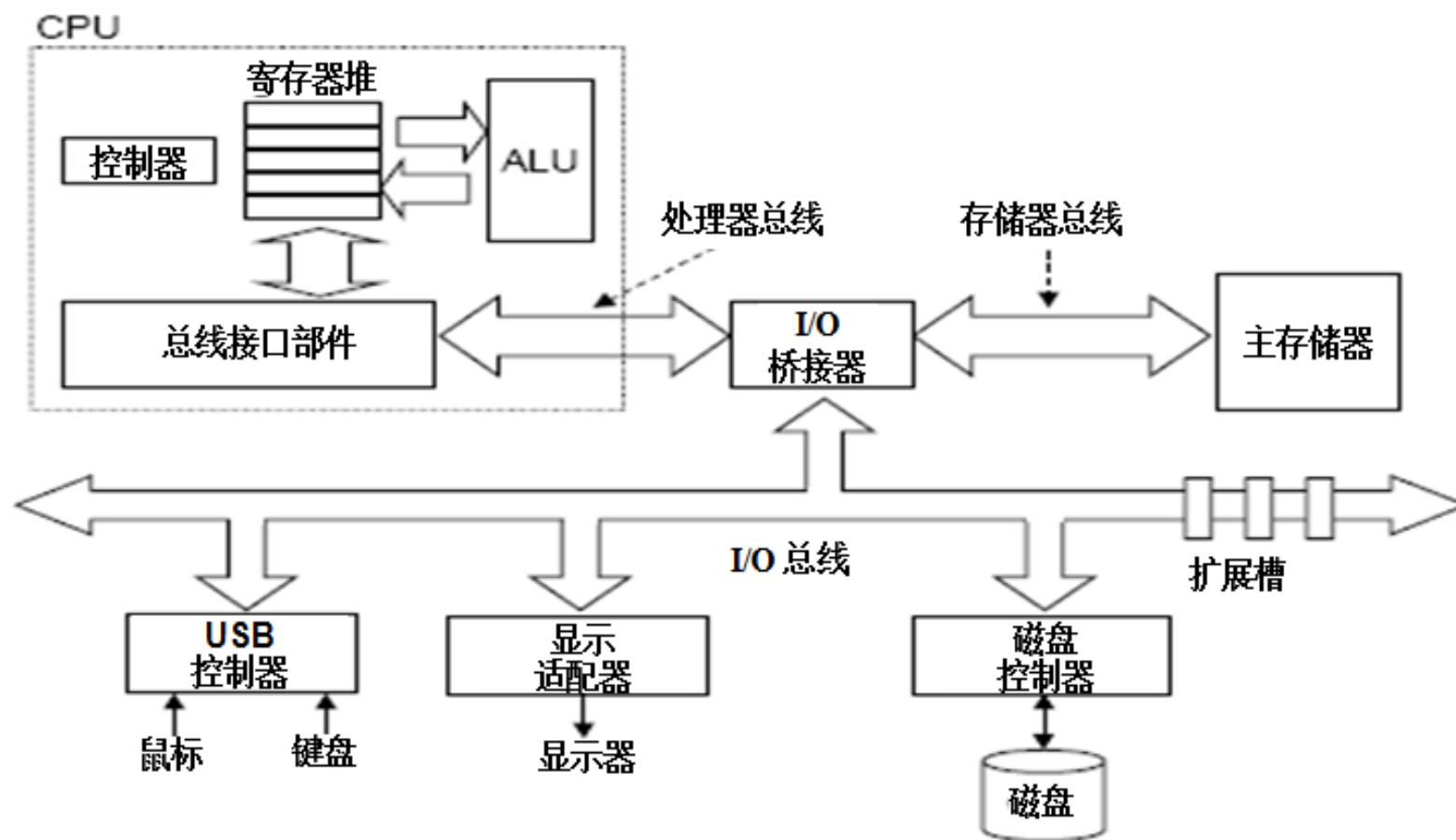
计算机科学与技术学院

哈尔滨工业大学（深圳）

主要内容

- Unix I/O
- 用RIO包健壮地读写
- 读取文件元数据，共享和重定位
- 标准I/O
- 结束语

回顾：计算机的体系结构



Unix I/O---与外部设备的交互方式

- 一个 Linux **文件** 就是一个 m 字节的序列:
 - $B_0, B_1, \dots, B_k, \dots, B_{m-1}$
- 现实情况: 所有的I/O设备都被模型化为文件:
 - **/dev/sda2** (用户磁盘分区) **文件是对IO设备的抽象**
 - **/dev/tty2** (终端)
- 甚至内核也被映射为文件:
 - **/boot/vmlinuz-3.13.0-55-generic** (内核映像)
 - **/proc** (内核数据结构)

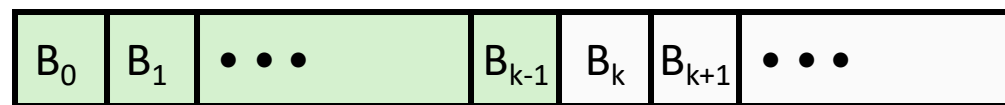
内核是操作系统代码常驻主存的部分。内核不是一个独立的进程,它是系统管理全部进程所用代码和数据结构的集合。

File Types 文件类型

- 每个Linux文件都有一个类型（**type**）来表明它在系统中的角色
- 文件系统抽象出的类型
 - 普通文件 (Regular file): 包含任意数据
 - 目录 (Directory): 一组链接文件的索引
- 其他类型
 - 命名通道 (*Named pipes (FIFOs)*)
 - 符号链接 (*Symbolic links*)
 - 字符和块设备 (*Character and block devices*)
 - 套接字 (**Socket**): 用来与另一个进程进行跨网络通信的文件

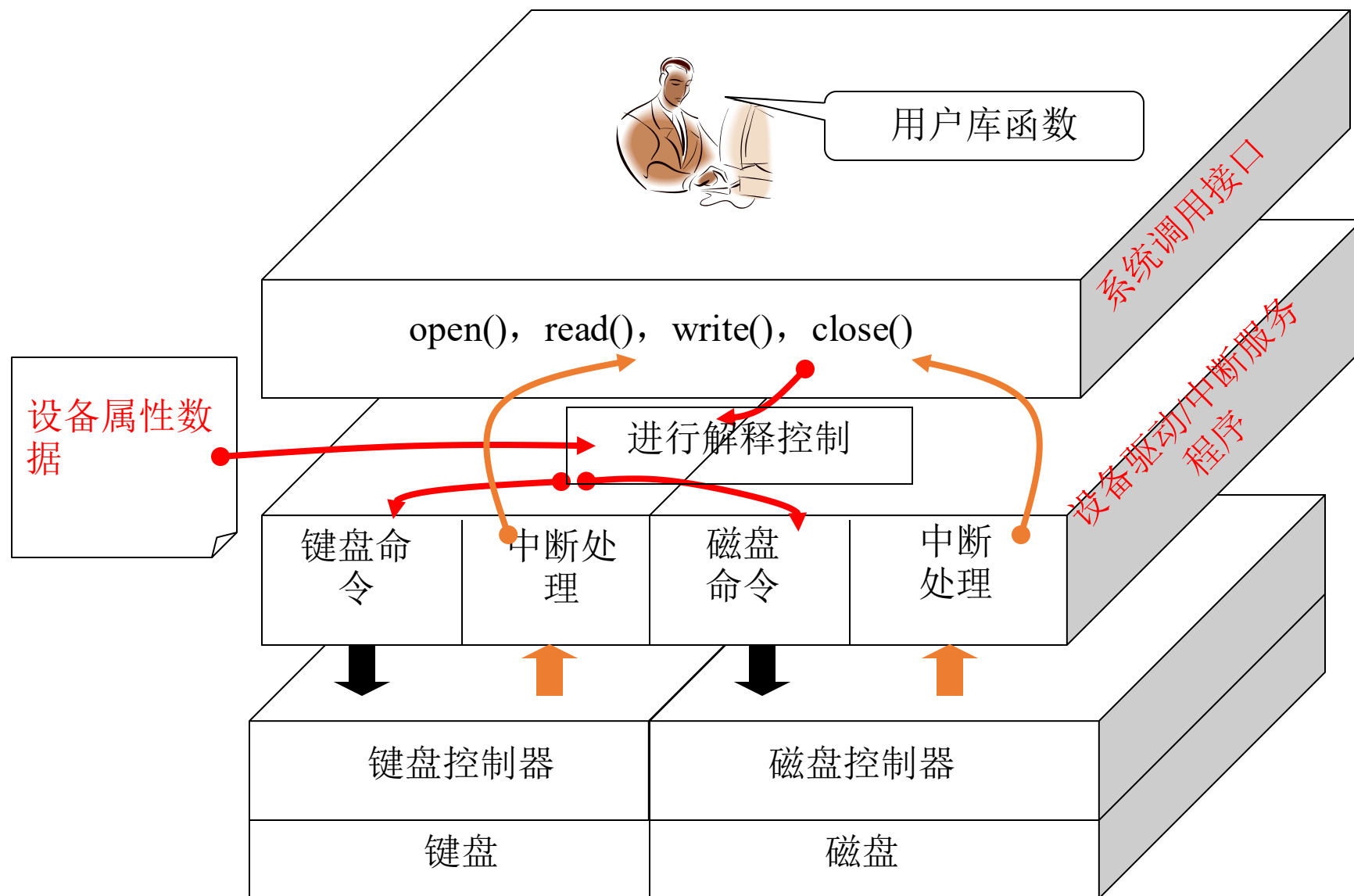
Unix I/O

- 将设备**映射为文件**的方式，允许Linux内核引出一个简单、低级、通用的接口，称为*Unix I/O*:
 - 打开和关闭文件
 - **open()** and **close()**
 - 读写文件
 - **read()** and **write()**
 - 改变**当前的文件位置** (seek)
 - 指示文件要读写位置的偏移量
 - **lseek()**



文件当前位置 = k

Unix 设备I/O的过程



I/O设备特点

属性	差异	例子
数据传输模式	字符、块	键盘、磁盘
访问方式	顺序、随机	光盘、SSD
传输方式	同步、异步	磁盘、磁带
分享	专用、共享	打印机、磁盘
设备速度	延迟、寻道时间、传输速率、操作延迟	-
I/O方向	只读、只写、读写	键盘、显示器

Linux中的设备

- cpu 处理器的msr寄存器
- block, char 块设备、字符设备
- full, null, zero 特殊设备
- stdin, stdout, stderr 标准输入/输出/报错
- random/urandom 随机数设备
- shm 共享内存
- sda, sdb, sdc, ..., sdh, nvme0, nvme1 磁盘

```
zxy@dell-r760:~$ ls /dev/
autofs      loop6       sdc         tty25       tty60       ttyS9
block       loop7       sdd         tty26       tty61       udmabuf
bsg         loop-control sde         tty27       tty62       uhid
btrfs-control mapper      sdf         tty28       tty63       uinput
bus         mcelog     sdg         tty29       tty7        urandom
char        md         sdh         tty3        tty8        userfaultfd
console     md0        sg0         tty30       tty9        userio
core        md1        sg1         tty31       ttyprintk   vcs
cpu         mem        sg2         tty32       ttyS0       vcs1
cpu_dma_latency mqueue     sg3         tty33       ttyS1       vcs2
cuse        mtd0       sg4         tty34       ttyS10      vcs3
disk        mtd0ro     sg5         tty35       ttyS11      vcs4
dma_heap    mtd1       sg6         tty36       ttyS12      vcs5
dri         mtd1ro     sg7         tty37       ttyS13      vcs6
ecryptfs    net        shm         tty38       ttyS14      vcsa
fb0         ng0n1      snapshot    tty39       ttyS15      vcsa1
fd          ng1n1      snd         tty4        ttyS16      vcsa2
full        null       stderr      tty40       ttyS17      vcsa3
fuse        nvme0      stdin       tty41       ttyS18      vcsa4
gpiochip0   nvme0n1    stdout      tty42       ttyS19      vcsa5
hpet        nvme0n1p1  tty        tty43       ttyS20      vcsa6
hugepages   nvme0n1p2  tty0        tty44       ttyS21      vcsu
hwrng       nvme1      tty1        tty45       ttyS22      vcsu1
i2c-0       nvme1n1    tty10       tty46       ttyS23      vcsu2
i2c-1       nvram      tty11       tty47       ttyS24      vcsu3
i2c-2       port       tty12       tty48       ttyS25      vcsu4
initctl     ppp        tty13       tty49       ttyS26      vcsu5
input       psaux      tty14       tty5        ttyS27      vcsu6
ipmi0       ptmx       tty15       tty50       ttyS28      vfbio
isst_interface ptp0       tty16       tty51       ttyS29      vga_arbiter
kmsg        ptp1       tty17       tty52       ttyS30      vhci
kvm         pts        tty18       tty53       ttyS31      vhost-net
log         qat_adf_ctl tty19       tty54       ttyS32      vhost-vsock
loop0       random     tty2        tty55       ttyS33      wmi
loop1       rfkill     tty20       tty56       ttyS34      zero
loop2       rtc        tty21       tty57       ttyS35
loop3       rtc0       tty22       tty58       ttyS36
loop4       sda        tty23       tty59       ttyS37
loop5       sdb        tty24       tty6        ttyS38
```

Linux中的设备

- cpu目录中保存了每个核的msr寄存器
- 每个核心有独立的msr寄存器
- msr的内容无法直接读取
- 记录CPU核心一系列设置和统计参数

```
[zxy@de11-r760:/dev]$ cd cpu/
[zxy@de11-r760:/dev/cpu]$ ls
0  11 14 17 2  22 25 28 30 33 36 39 41 44 47 5  52 55 58 60 63 66 69 71 74 77 8  82 85 88 90 93
1  12 15 18 20 23 26 29 31 34 37 4  42 45 48 50 53 56 59 61 64 67 7  72 75 78 80 83 86 89 91 94
10 13 16 19 21 24 27 3  32 35 38 40 43 46 49 51 54 57 6  62 65 68 70 73 76 79 81 84 87 9  92 95
[zxy@de11-r760:/dev/cpu]$ cd 0/
[zxy@de11-r760:/dev/cpu/0]$ ls
msr
[zxy@de11-r760:/dev/cpu/0]$ sudo rdmsr -p 0 0x10  ← 启动以来的时钟周期数
695094bd2c211
[zxy@de11-r760:/dev/cpu/0]$ sudo rdmsr -p 0 0x1B0 ← 能效偏好（越大越偏向节能）
6
[zxy@de11-r760:/dev/cpu/0]$ sudo rdmsr -p 0 0x19C ← 温度控制参数
88200800
[zxy@de11-r760:/dev/cpu/0]$
```

Linux中的设备

- block目录中保存了所有块设备
- 每一项都是具体设备的软链接
 - 类似于windows中的“快捷方式”
- 主要包括
 - SATA接口硬盘 (sda, sdb, ...)
 - NVME接口硬盘 (nvme1n1, ...)
 - 镜像 (loop)
 - RAID (md)

```
zxy@dell-r760:/dev$ cd block/
zxy@dell-r760:/dev/block$ ls
259:0 259:1 259:2 259:3 7:0 7:1 7:2 7:3 7:4 7:5 7:6 7:
7 8:0 8:112 8:16 8:32 8:48 8:64 8:80 8:96 9:0 9:1
zxy@dell-r760:/dev/block$ ls -al
total 0
drwxr-xr-x  2 root root  480 Oct 27 08:05 .
drwxr-xr-x 21 root root 4640 Oct 27 08:03 ..
lrwxrwxrwx  1 root root   10 Oct 27 08:03 259:0 -> ../nvme1n1
lrwxrwxrwx  1 root root   10 Oct 27 08:03 259:1 -> ../nvme0n1
lrwxrwxrwx  1 root root   12 Oct 27 08:03 259:2 -> ../nvme0n1p1
lrwxrwxrwx  1 root root   12 Oct 27 08:03 259:3 -> ../nvme0n1p2
lrwxrwxrwx  1 root root    8 Oct 27 08:03 7:0 -> ../loop0
lrwxrwxrwx  1 root root    8 Oct 27 08:03 7:1 -> ../loop1
lrwxrwxrwx  1 root root    8 Oct 27 08:03 7:2 -> ../loop2
lrwxrwxrwx  1 root root    8 Oct 27 08:03 7:3 -> ../loop3
lrwxrwxrwx  1 root root    8 Oct 27 08:05 7:4 -> ../loop4
lrwxrwxrwx  1 root root    8 Oct 27 08:03 7:5 -> ../loop5
lrwxrwxrwx  1 root root    8 Oct 27 08:03 7:6 -> ../loop6
lrwxrwxrwx  1 root root    8 Oct 27 08:03 7:7 -> ../loop7
lrwxrwxrwx  1 root root    6 Oct 27 08:03 8:0 -> ../sda
lrwxrwxrwx  1 root root    6 Oct 27 08:03 8:112 -> ../sdh
lrwxrwxrwx  1 root root    6 Oct 27 08:03 8:16 -> ../sdb
lrwxrwxrwx  1 root root    6 Oct 27 08:03 8:32 -> ../sdc
lrwxrwxrwx  1 root root    6 Oct 27 08:03 8:48 -> ../sdd
lrwxrwxrwx  1 root root    6 Oct 27 08:03 8:64 -> ../sde
lrwxrwxrwx  1 root root    6 Oct 27 08:03 8:80 -> ../sdf
lrwxrwxrwx  1 root root    6 Oct 27 08:03 8:96 -> ../sdg
lrwxrwxrwx  1 root root    6 Oct 27 08:03 9:0 -> ../md0
lrwxrwxrwx  1 root root    6 Oct 27 08:03 9:1 -> ../md1
zxy@dell-r760:/dev/block$
```


Linux中的设备

- char目录中保存了所有字符设备
- 每一项都是具体设备的软链接
- 主要包括
 - 网卡 (tun, ...)
 - 用户层文件系统接口 (fuse)
 - 内存 (mem)
 - 随机数发生器 (random/urandom)
 - 终端连接 (tty, pty)
 - ...

```
zxy@de11-r760:/dev$ cd char/
zxy@de11-r760:/dev/char$ ls
10:1 10:231 1:4 202:12 202:25 202:38 202:50 202:63 202:76 202:89 21:6 243:5 4:13 4:26 4:39 4:51 4:64 4:77 4:9 7:129 7:67
10:121 10:232 1:5 202:13 202:26 202:39 202:51 202:64 202:77 202:9 21:7 243:6 4:14 4:27 4:4 4:52 4:65 4:78 4:90 7:130 7:68
10:122 10:234 1:7 202:14 202:27 202:4 202:52 202:65 202:78 202:90 226:0 243:7 4:15 4:28 4:40 4:53 4:66 4:79 4:91 7:131 7:69
10:123 10:235 1:8 202:15 202:28 202:40 202:53 202:66 202:79 202:91 235:0 246:0 4:16 4:29 4:41 4:54 4:67 4:8 4:92 7:132 7:70
10:124 10:236 189:0 202:16 202:29 202:41 202:54 202:67 202:8 202:92 237:0 246:1 4:17 4:3 4:42 4:55 4:68 4:80 4:93 7:133 89:0
10:125 10:237 189:1 202:17 202:3 202:42 202:55 202:68 202:80 202:93 239:0 248:0 4:18 4:30 4:43 4:56 4:69 4:81 4:94 7:134 89:1
10:126 10:242 189:128 202:18 202:30 202:43 202:56 202:69 202:81 202:94 239:1 249:0 4:19 4:31 4:44 4:57 4:7 4:82 4:95 7:2 89:2
10:127 108:0 189:2 202:19 202:31 202:44 202:57 202:7 202:82 202:95 240:0 254:0 4:2 4:32 4:45 4:58 4:70 4:83 5:0 7:3 90:0
10:183 1:1 189:3 202:2 202:32 202:45 202:58 202:70 202:83 21:0 240:1 29:0 4:20 4:33 4:46 4:59 4:71 4:84 5:1 7:4 90:1
10:200 1:11 1:9 202:20 202:33 202:46 202:59 202:71 202:84 21:1 243:0 4:0 4:21 4:34 4:47 4:6 4:72 4:85 5:2 7:5 90:2
10:223 1:3 202:0 202:21 202:34 202:47 202:6 202:72 202:85 21:2 243:1 4:1 4:22 4:35 4:48 4:60 4:73 4:86 5:3 7:6 90:3
10:227 13:63 202:1 202:22 202:35 202:48 202:60 202:73 202:86 21:3 243:2 4:10 4:23 4:36 4:49 4:61 4:74 4:87 7:0 7:64
10:228 13:64 202:10 202:23 202:36 202:49 202:61 202:74 202:87 21:4 243:3 4:11 4:24 4:37 4:5 4:62 4:75 4:88 7:1 7:65
10:229 13:65 202:11 202:24 202:37 202:5 202:62 202:75 202:88 21:5 243:4 4:12 4:25 4:38 4:50 4:63 4:76 4:89 7:128 7:66
zxy@de11-r760:/dev/char$ ls -al
total 0
drwxr-xr-x 2 root root 5860 Oct 27 08:03 .
drwxr-xr-x 21 root root 4640 Oct 27 08:03 ..
lrwxrwxrwx 1 root root 8 Oct 27 08:03 10:1 -> ../psaux
lrwxrwxrwx 1 root root 18 Oct 27 08:03 10:121 -> ../wmi/dell-smbios
lrwxrwxrwx 1 root root 17 Oct 27 08:03 10:122 -> ../isst_interface
lrwxrwxrwx 1 root root 18 Oct 27 08:03 10:123 -> ../cpu_dma_latency
lrwxrwxrwx 1 root root 10 Oct 27 08:03 10:124 -> ../udmabuf
lrwxrwxrwx 1 root root 11 Oct 27 08:03 10:125 -> ../ecryptfs
lrwxrwxrwx 1 root root 14 Oct 27 08:03 10:126 -> ../userfaultfd
lrwxrwxrwx 1 root root 14 Oct 27 08:03 10:127 -> ../vga_arbiter
lrwxrwxrwx 1 root root 8 Oct 27 08:03 10:183 -> ../hwrng
lrwxrwxrwx 1 root root 10 Oct 27 08:03 10:200 -> ../net/tun
lrwxrwxrwx 1 root root 9 Oct 27 08:03 10:223 -> ../input
lrwxrwxrwx 1 root root 9 Oct 27 08:03 10:227 -> ../mcelog
lrwxrwxrwx 1 root root 7 Oct 27 08:03 10:228 -> ../hpet
lrwxrwxrwx 1 root root 7 Oct 27 08:03 10:229 -> ../fuse
lrwxrwxrwx 1 root root 11 Oct 27 08:03 10:231 -> ../snapshot
lrwxrwxrwx 1 root root 6 Oct 27 08:03 10:232 -> ../kvm
lrwxrwxrwx 1 root root 16 Oct 27 08:03 10:234 -> ../btrfs-control
lrwxrwxrwx 1 root root 9 Oct 27 08:03 10:235 -> ../autofs
lrwxrwxrwx 1 root root 17 Oct 27 08:03 10:236 -> ../mapper/control
lrwxrwxrwx 1 root root 15 Oct 27 08:03 10:237 -> ../loop-control
lrwxrwxrwx 1 root root 9 Oct 27 08:03 10:242 -> ../rfkill
lrwxrwxrwx 1 root root 6 Oct 27 08:03 108:0 -> ../ppp
lrwxrwxrwx 1 root root 6 Oct 27 08:03 1:1 -> ../mem
lrwxrwxrwx 1 root root 7 Oct 27 08:03 1:11 -> ../kmsd
lrwxrwxrwx 1 root root 7 Oct 27 08:03 1:3 -> ../null
```

```
zxy@de11-r760:/dev/char$ sudo head -c 16 1:1 | xxd -p
```

```
000000000000000006ada00f0d8dd00f0
```

物理内存前16个字节

Linux中的设备

- full 写入数据总是返回空间满
- null “垃圾桶”
- zero 读取数据总是返回0
- stdin, stdout, stderr 标准输入/输出/报错
- random/urandom 随机数生成器
 - “真正的”随机数
 - 通过收集噪声和熵源，从硬件或操作系统事件中采样
 - random可能会阻塞

```
[zxy@dell-r760:/dev$ echo "1" > full
-bash: echo: write error: No space left on device
[zxy@dell-r760:/dev$ echo "1" > null
[zxy@dell-r760:/dev$ head -c 16 /dev/zero | xxd -p
00000000000000000000000000000000
[zxy@dell-r760:/dev$ echo "1" > stdout
1
[zxy@dell-r760:/dev$ head -c 16 /dev/random | xxd -p
9b415daddb9f7408a09c6480dcc386f0
[zxy@dell-r760:/dev$ head -c 16 /dev/random | xxd -p
6d786c359a3340bb6bb81a54d5c1318e
zxy@dell-r760:/dev$
```

Linux中的设备

- shm 共享内存 (shared memory)
- 对应内存中共享区域
- 进程间通信方式之一
- 临时文件系统tmpfs

```
[zxy@dell-r760:/dev$ cd shm/
[zxy@dell-r760:/dev/shm$ ls
[zxy@dell-r760:/dev/shm$ free -h
```

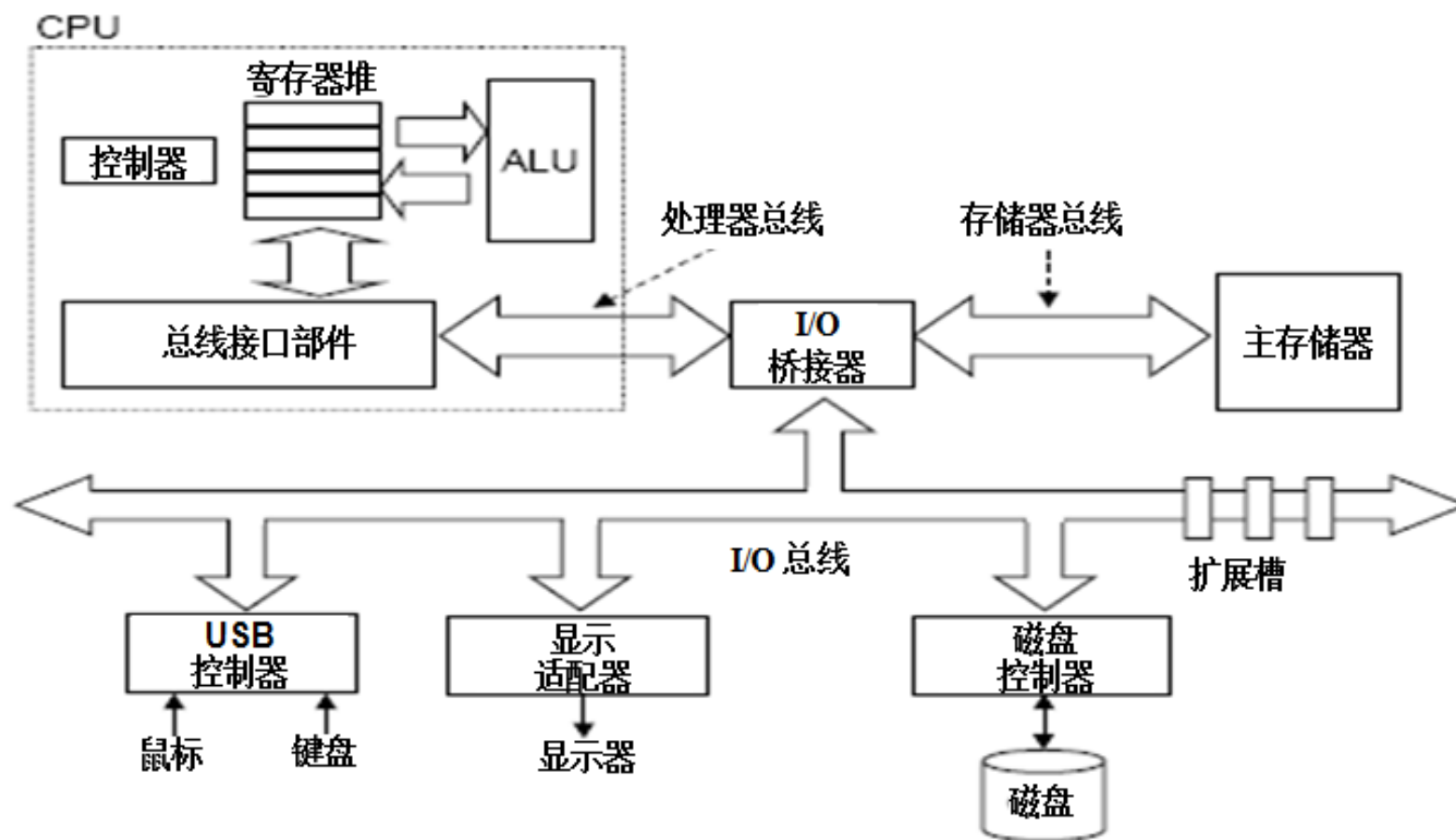
	total	used	free	shared	buff/cache	available
Mem:	125Gi	2.0Gi	43Gi	4.0Mi	79Gi	121Gi
Swap:	8.0Gi	328Mi	7.7Gi			

```
[zxy@dell-r760:/dev/shm$ cp /mnt/nvme-1/CHM/82.0.4066.0.tar .
[zxy@dell-r760:/dev/shm$ ls -alh
total 3.0G
drwxrwxrwt  2 root root  60 Nov  5 07:07 .
drwxr-xr-x 21 root root 4.8K Oct 23 12:14 ..
-rw-rw-r--  1 zxy  zxy  3.0G Nov  5 07:07 82.0.4066.0.tar
[zxy@dell-r760:/dev/shm$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	125Gi	2.0Gi	40Gi	2.9Gi	82Gi	119Gi
Swap:	8.0Gi	328Mi	7.7Gi			

```
zxy@dell-r760:/dev/shm$
```

回顾：计算机的体系结构



Regular Files 普通文件

- 普通文件包含任意数据
- 应用程序常常要区分文本文件 (*text files*) 和二进制文件 (*binary files*)
 - 文本文件是只包含 ASCII 或 Unicode 字符的普通文件
 - 二进制文件是所有其他文件
 - 比如 目标文件, JPEG 图像文件等等
 - 内核并不知道两者之间的区别
- Linux 文本文件是文本行的序列
 - 文本行是一个字符序列, 以一个新行符 (`'\n'`) 结束
 - 新行符为 `0xa`, 与 ASCII 的换行符 (LF) 是一样的
- 其他系统中的行结束标志
 - Linux 和 Mac 操作系统: `'\n'` (`0xa`)
 - 换行 (LF, line feed)
 - Windows 和 因特网络协议: `'\r\n'` (`0xd 0xa`)
 - Carriage return (CR) followed by line feed (LF)
回车换行



目录是一种特殊的文件

```
[→ Workplace ls -al log
```

```
total 800
```

```
drwxr-xr-x 13 zouxiangyu staff 416 11 12 22:57 .
drwxr-xr-x@ 11 zouxiangyu staff 352 11 12 22:56 ..
drwxr-xr-x 14 zouxiangyu staff 448 7 16 18:07 MeGA
drwxr-xr-x 13 zouxiangyu staff 416 7 10 21:46 MeGA_0509
drwxr-xr-x 13 zouxiangyu staff 416 7 10 21:46 MeGA_0511
drwxr-xr-x 12 zouxiangyu staff 384 7 21 23:50 MeGA_10_1
drwxr-xr-x 12 zouxiangyu staff 384 7 17 22:50 MeGA_10_m
drwxr-xr-x 12 zouxiangyu staff 384 7 14 15:51 MeGA_40_1
drwxr-xr-x 12 zouxiangyu staff
drwxr-xr-x 2 zouxiangyu staff
drwxr-xr-x 5 zouxiangyu staff
-rw-r--r--@ 1 zouxiangyu staff 3
-rw-r--r--@ 1 zouxiangyu staff 37
```

```
[→ Workplace vim log
```

```
→ Workplace
```

Workplace — vim log — vim — vim log — 117x30

```
"
=====
" Netrw Directory Listing                                (netrw v173)
" /Users/zouxiangyu/Workplace/log
" Sorted by      name
" Sort sequence: [V]$, \<core\%(\\.d\\+\\)\=\\>, \.h$, \.c$, \.cpp$, \~=\*$, *, \.o$, \.obj$, \.info$, \.swp$, \.bak$, \~$
" Quick Help: <F1>:help  -:go up dir  D:delete  R:rename  s:sort-by  x:special
"
=====
```

```
./
./
MeGA/
MeGA_0509/
MeGA_0511/
MeGA_10_1/
MeGA_10_m/
MeGA_40_1/
MeGA_40_m/
MeGA_Seg/
MeGA_newDataset/
specratio.chm
specratio.web
```

```
~
~
~
~
~
~
```

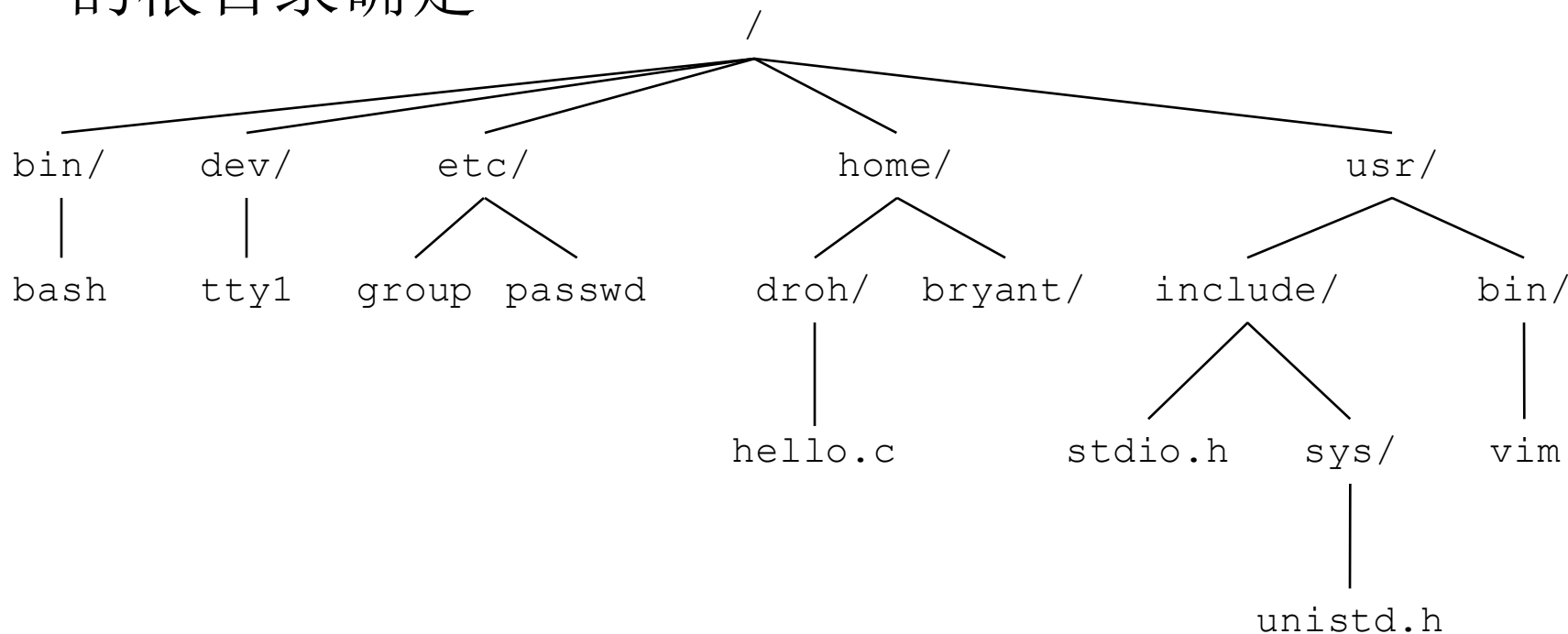
```
"log" is a directory
```

Directories 目录

- 目录包含一组链接
 - 每个链接将一个文件名映射到一个文件
- 每个目录至少含有两个条目
 - `.` 是到该文件自身的链接
 - `..` 是到目录层次结构中父目录的链接
- 操作目录命令
 - `mkdir`: 创建空目录
 - `ls`: 查看目录内容
 - `rmdir`: 删除空目录

Directory Hierarchy 目录层次结构

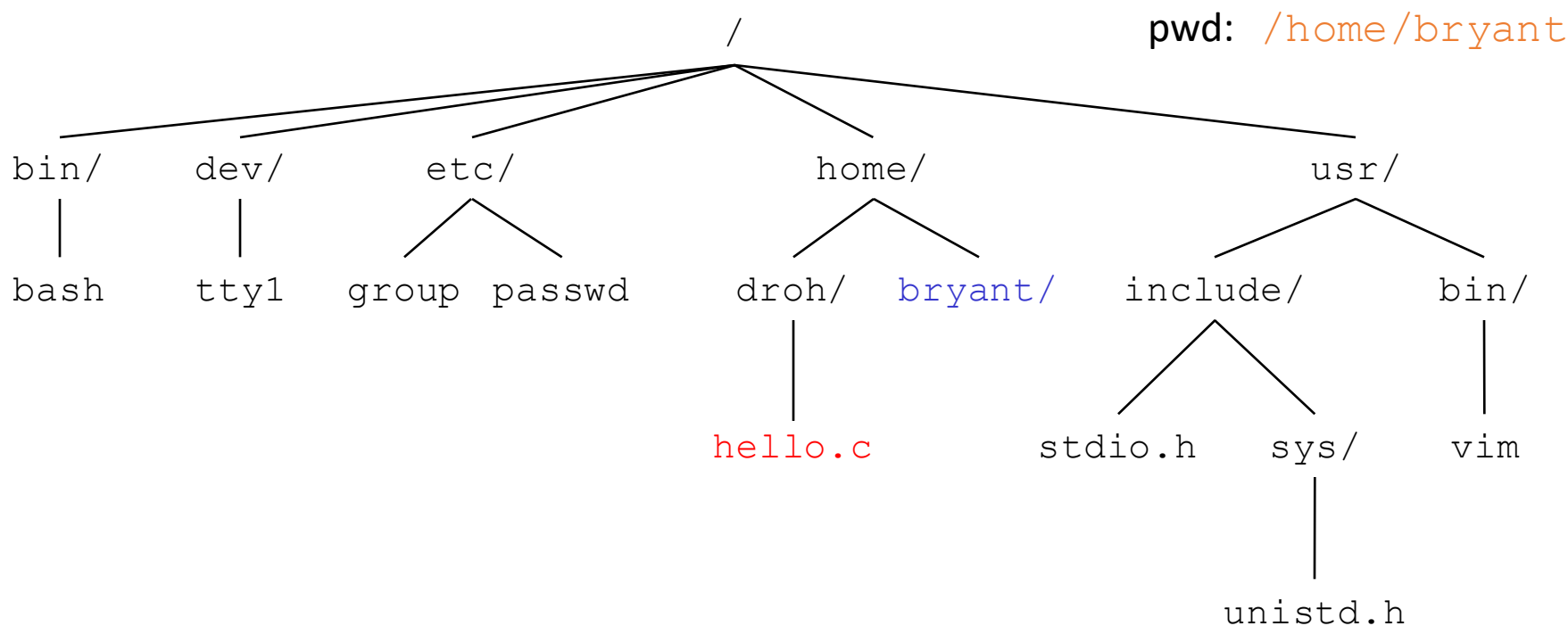
- 所有文件都组织成一个目录层次结构，由名为 / (斜杠) 的根目录确定



- 内核为每个进程都保存着一个当前工作目录 (*current working directory (cwd)*)
 - 可以用cd命令来修改shell中的当前工作目录

Pathnames 路径名

- 目录层次结构中的位置用*路径名*来指定
 - *绝对路径名* 以 ‘/’ 开始，表示从根节点开始的路径
 - /home/droh/hello.c
 - *相对路径名* 以文件名开始，表示从当前工作目录开始的路径
 - ../home/droh/hello.c



Opening Files 打开文件

- 打开文件是通知内核你准备好访问该文件

```
int fd;    /* file descriptor */

if ((fd = open("/etc/hosts", O_RDONLY)) < 0) {
    perror("open");
    exit(1);
}
```

- 返回一个小的描述符数字---- **文件描述符**。返回的描述符总是在进程中当前没有打开的最小描述符。
 - **fd == -1** 说明发生错误
- Linux内核创建的每个进程都以与一个终端相关联的三个虚拟设备开始:
 - 0: 标准输入 (stdin)
 - 1: 标准输出 (stdout)
 - 2: 标准错误 (stderr)

Closing Files 关闭文件

- 关闭文件是通知内核你要结束访问一个文件

```
int fd;      /* file descriptor */
int retval; /* return value */

if ((retval = close(fd)) < 0) {
    perror("close");
    exit(1);
}
```

- 关闭一个已经关闭的文件是导致线程程序灾难的一个因素
- 好习惯: 总是检查返回码, 即使是看似良性的函数, 比如 `close()`

Reading Files 读文件

- 读文件从当前文件位置复制字节到内存位置，然后更新文件位置

```
char buf[512];
int fd;          /* file descriptor */
int nbytes;      /* number of bytes read */

/* Open file fd ... */
/* Then read up to 512 bytes from file fd */
if ((nbytes = read(fd, buf, sizeof(buf))) < 0) {
    perror("read");
    exit(1);
}
```

- 返回值表示的是实际传送的字节数量
 - 返回类型 **ssize_t** 是有符号整数
 - **nbytes < 0** 表示发生错误
 - **不足值 (Short counts)** (**nbytes < sizeof(buf)**) 是可能的，**不是错误!**

Writing Files 写文件

- 写文件从内存复制字节到当前文件位置，然后更新文件位置

```
char buf[512];
int fd;          /* file descriptor */
int nbytes;      /* number of bytes write */

/* Open the file fd ... */
/* Then write up to 512 bytes from buf to file fd */
if ((nbytes = write(fd, buf, sizeof(buf))) < 0) {
    perror("write");
    exit(1);
}
```

- 返回值表示的是从内存向文件fd实际传送的字节数量
 - **nbytes < 0** 表明发生错误
 - 同读文件一样, 不足值 (short counts) 是可能的, 并不是错误!

简单Unix I/O示例

- 一次一个字节地从标准输入复制到标准输出

```
#include "csapp.h"

int main(void)
{
    char c;

    while (Read(STDIN_FILENO, &c, 1) != 0)
        Write(STDOUT_FILENO, &c, 1);
    exit(0);
}
```

On Short Counts 不足值

- 出现“不足值”的几种情况：
 - Encountering (end-of-file) EOF on reads 读时遇到EOF
 - Reading text lines from a terminal 从终端读文本行
 - Reading and writing network sockets 读写网络套接字
- 以下几种情况不会出现“不足值”：
 - 正常的读磁盘文件（遇到 EOF 出现不足值）
 - 正常的写磁盘文件（磁盘满也会出现不足值）
- 最好的解决办法就是一直允许不足值，反复处理不足值
 - 反复调用read和write处理不足值，直到所有需要的字节都传送完毕

主要内容

- Unix I/O
- 用RIO包健壮地读写
- 读取文件元数据，共享和重定位
- 标准I/O
- 结束语

The RIO Package RIO包（Robust）

- 如何妥善的解决不足值问题？以RIO为例
- RIO提供两类不同的函数
 - 无缓冲的输入输出函数 Unbuffered input and output of binary data
 - **rio_readn**和 **rio_writen**
 - 带缓冲的输入函数 Buffered input of text lines and binary data
 - **rio_readlineb** 和 **rio_readnb**
 - 带缓冲的 RIO 输入函数是线程安全的，它在同一个描述符上可以被交错地调用
- 下载地址： <http://csapp.cs.cmu.edu/3e/code.html>
→ **src/csapp.c** and **include/csapp.h**

RIO的无缓冲的输入输出函数

- 使用与 `Unix read` 和 `write` 相同的接口
- 对于在网络套接字上传输数据特别有用

```
#include "csapp.h"
```

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n);  
ssize_t rio_writen(int fd, void *usrbuf, size_t n);
```

Return: num. bytes transferred if OK, 0 on EOF (`rio_readn` only), -1 on error

- **`rio_readn`** 在遇到 EOF 时只能返回一个不足值
 - 只有当确定读取字节数时才使用它
- **`rio_writen`** 绝不会返回不足值
- 对同一个描述符，可以任意交错地调用 **`rio_readn`** 和 **`rio_writen`**

rio_readn函数

```
/*
 * rio_readn - Robustly read n bytes (unbuffered)
 */
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* Interrupted by sig handler return */
                nread = 0;      /* and call read() again */
            else
                return -1;      /* errno set by read() */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);        /* Return >= 0 */
}
```

RIO的带缓冲的输入函数

- 高效地从内部内存缓冲区中缓存的文件中读取文本行和二进制数据

```
#include "csapp.h"
```

```
void rio_readinitb(rio_t *rp, int fd);
```

```
ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen);
```

```
ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n);
```

Return: num. bytes read if OK, 0 on EOF, -1 on error

- **rio_readlineb** 从文件fd中读取最大长度文本行，并存储在**usrbuf**
 - 对于从网络套接字上读取文本行特别有用
- 停止条件
 - 已经读了最大字节数
 - 遇到EOF
 - 遇到新行符 ('\n')

RIO的带缓冲的输入函数

```
#include "csapp.h"
//将描述符fd和地址rp处的一个类型为rio_t的读缓冲区联系起来
void rio_readinitb(rio_t *rp, int fd);

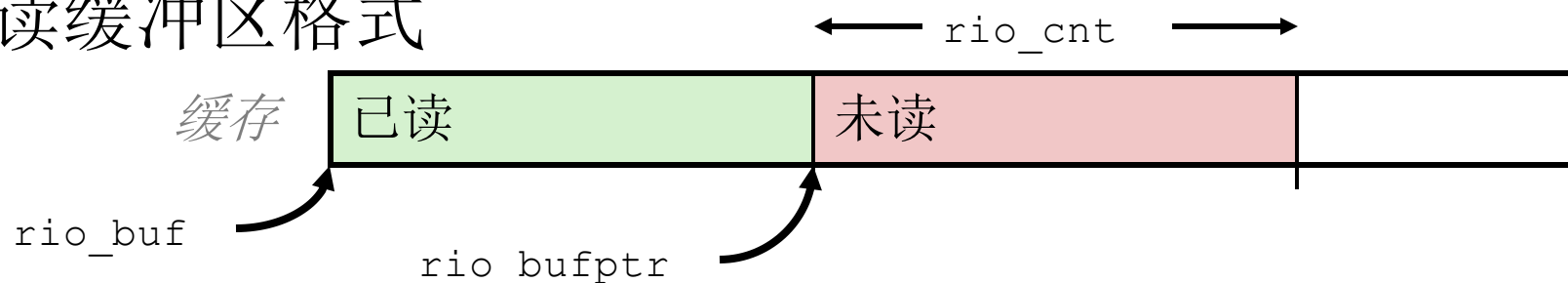
ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen);
ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n);

Return: num. bytes read if OK, 0 on EOF, -1 on error
```

- **rio_readnb** 从文件 **fd** 最多读**n**个字节
- 停止条件
 - 已读最大字节数 **maxlen**
 - 遇到EOF
- 同一个描述符对 **rio_readlineb** 和 **rio_readnb** 的调用可以任意交叉进行
 - 警告: 不要和 **rio_readn** 函数交叉使用

Buffered I/O: Declaration 带缓存的IO: 声明

- 读缓冲区格式



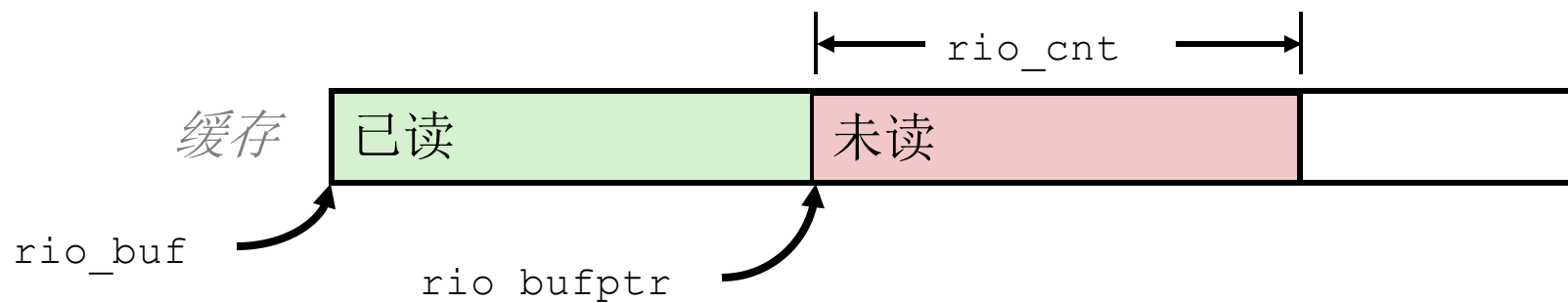
```
typedef struct {  
    int rio_fd; /* descriptor for this internal buf */  
    int rio_cnt; /* unread bytes in internal buf */  
    char *rio_bufptr; /* next unread byte in internal buf */  
    char rio_buf[RIO_BUFSIZE]; /* internal buffer */  
} rio_t;
```

■ rio_readinitb函数

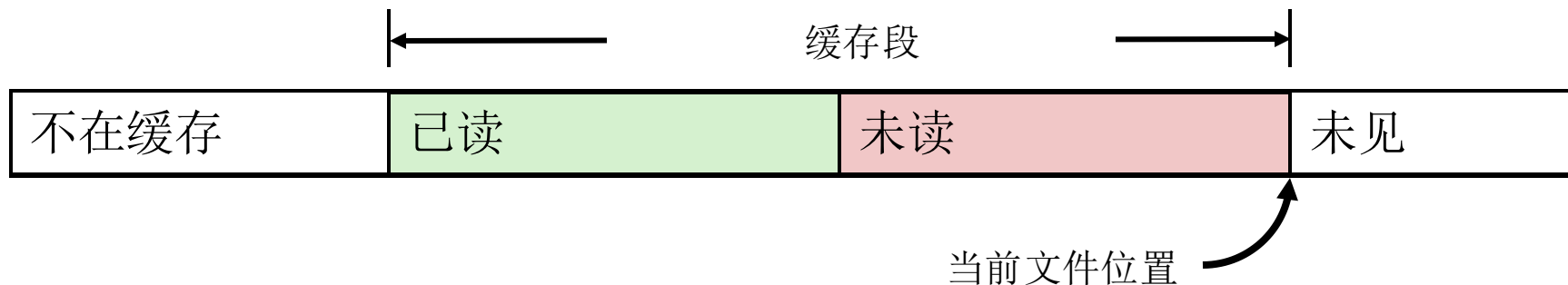
```
void rio_readinitb(rio_t *rp, int fd)  
{  
    rp->rio_fd = fd;  
    rp->rio_cnt = 0;  
    rp->rio_bufptr = rp->rio_buf;  
}
```

Buffered I/O: Implementation 带缓冲I/O的应用

- 读文件
- 文件有关联的缓冲区来保存从文件中读取，但是还未被用户代码读取的字节



- Unix文件分层:



RIO函数示例

- 从标准输入复制一个文本文件到标准输出

```
#include "csapp.h"

int main(int argc, char **argv)
{
    int n;
    rio_t rio;
    char buf[MAXLINE];

    Rio_readinitb(&rio, STDIN_FILENO);
    while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0)
        Rio_writen(STDOUT_FILENO, buf, n);
    exit(0);
}
```

cpfile.c

主要内容

- Unix I/O
- 用RIO包健壮地读写
- 读取文件元数据，共享和重定位
- 标准I/O
- 结束语

File Metadata 读取文件元数据

- *元数据 (Metadata)* 是关于文件的信息
- 每个文件的元数据都由内核来保存
 - 用户通过调用 **stat**和**fstat** 函数访问元数据

```
/* Metadata returned by the stat and fstat functions */
struct stat {
    dev_t      st_dev;      /* Device */
    ino_t      st_ino;      /* inode */
    mode_t     st_mode;     /* Protection and file type */
    nlink_t    st_nlink;    /* Number of hard links */
    uid_t      st_uid;      /* User ID of owner */
    gid_t      st_gid;      /* Group ID of owner */
    dev_t      st_rdev;     /* Device type (if inode device) */
    off_t      st_size;     /* Total size, in bytes */
    unsigned long st_blksize; /* Blocksize for filesystem I/O */
    unsigned long st_blocks; /* Number of blocks allocated */
    time_t     st_atime;    /* Time of last access */
    time_t     st_mtime;    /* Time of last modification */
    time_t     st_ctime;    /* Time of last change */
};
```

访问文件元数据示例

```
int main (int argc, char **argv)
{
    struct stat stat;
    char *type, *readok;

    Stat(argv[1], &stat);
    if (S_ISREG(stat.st_mode))          /* Determine file type */
        type = "regular";
    else if (S_ISDIR(stat.st_mode))
        type = "directory";
    else
        type = "other";
    if ((stat.st_mode & S_IRUSR)) /* Check read access */
        readok = "yes";
    else
        readok = "no";

    printf("type: %s, read: %s\n", type, readok);
    exit(0);
}
```

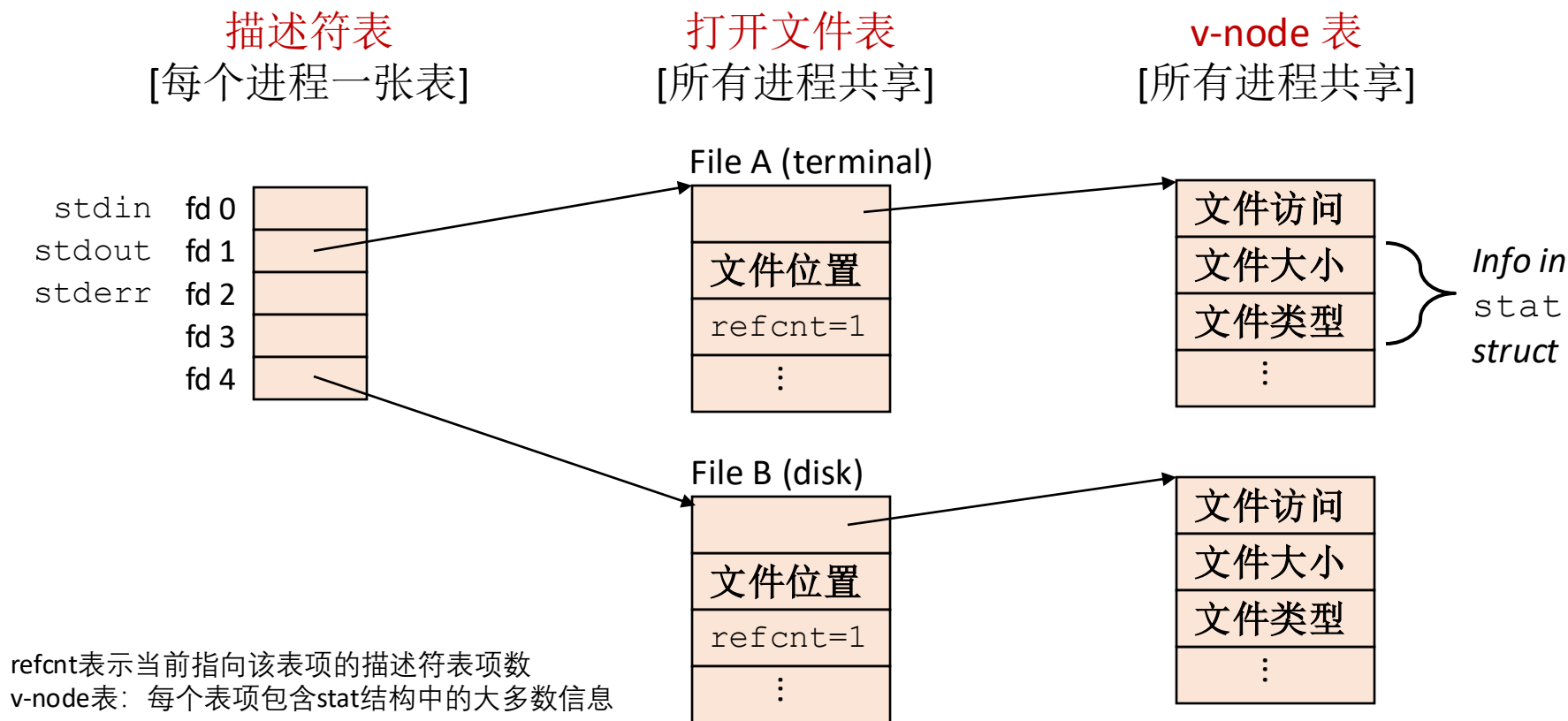
statcheck.c

```
linux> ./statcheck statcheck.c
type: regular, read: yes
linux> chmod 000 statcheck.c
linux> ./statcheck statcheck.c
type: regular, read: no
linux> ./statcheck ..
type: directory, read: yes
```

Unix内核如何表示打开文件

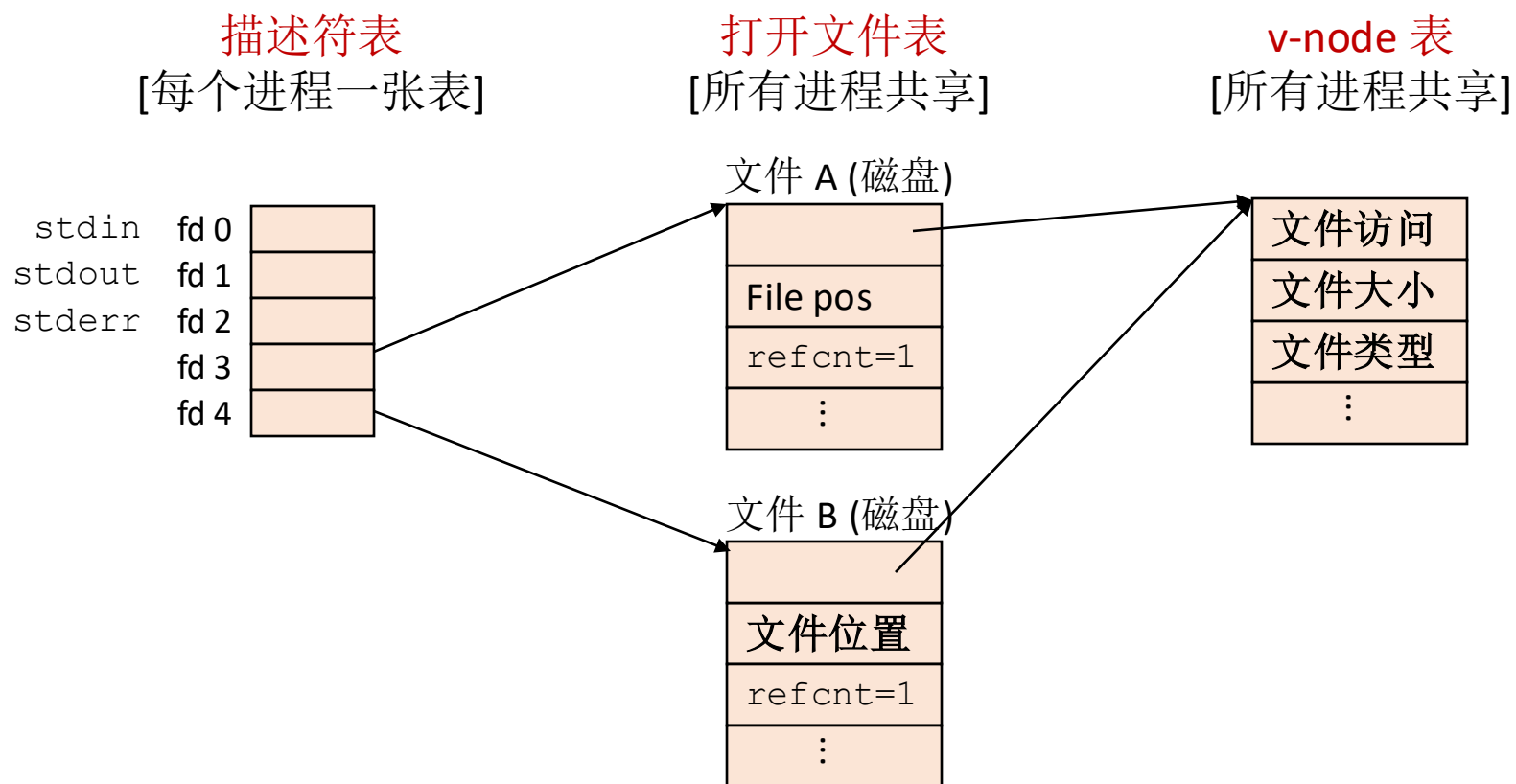
- 两个描述符引用两个不同的打开文件（没有共享）

描述符 1 (stdout) 指向终端, 描述符 4 指向打开磁盘文件



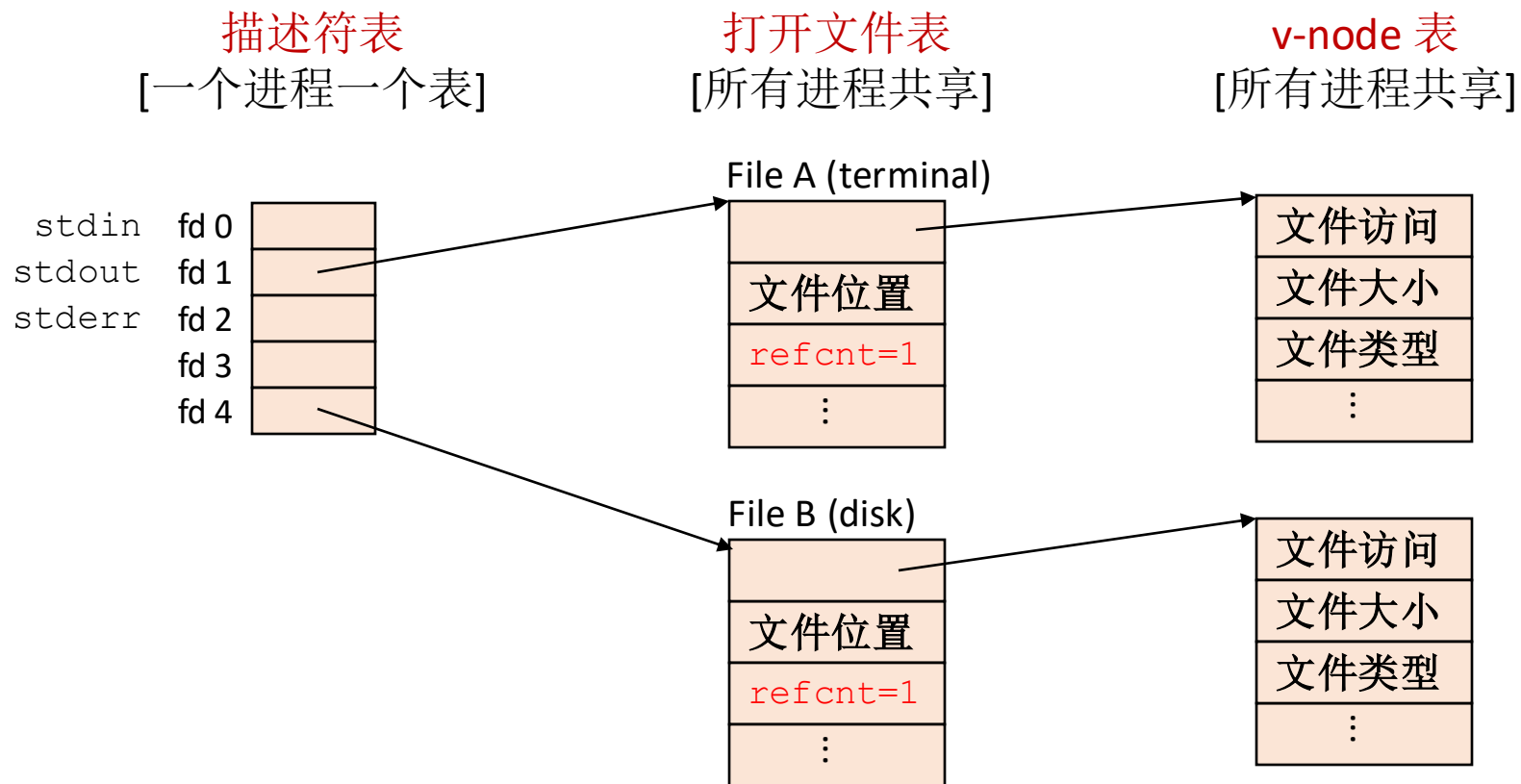
File Sharing 共享文件

- 两个不同的描述符通过两个不同的打开文件表表项来共享同一个磁盘文件
 - 例如，以同一个filename调用open函数两次



进程如何共享文件: fork

- 子进程继承父进程的打开文件
 - 注意: **共享相同的文件位置** (使用 **fcntl** 改变位置)
- 调用 `fork` 之前:



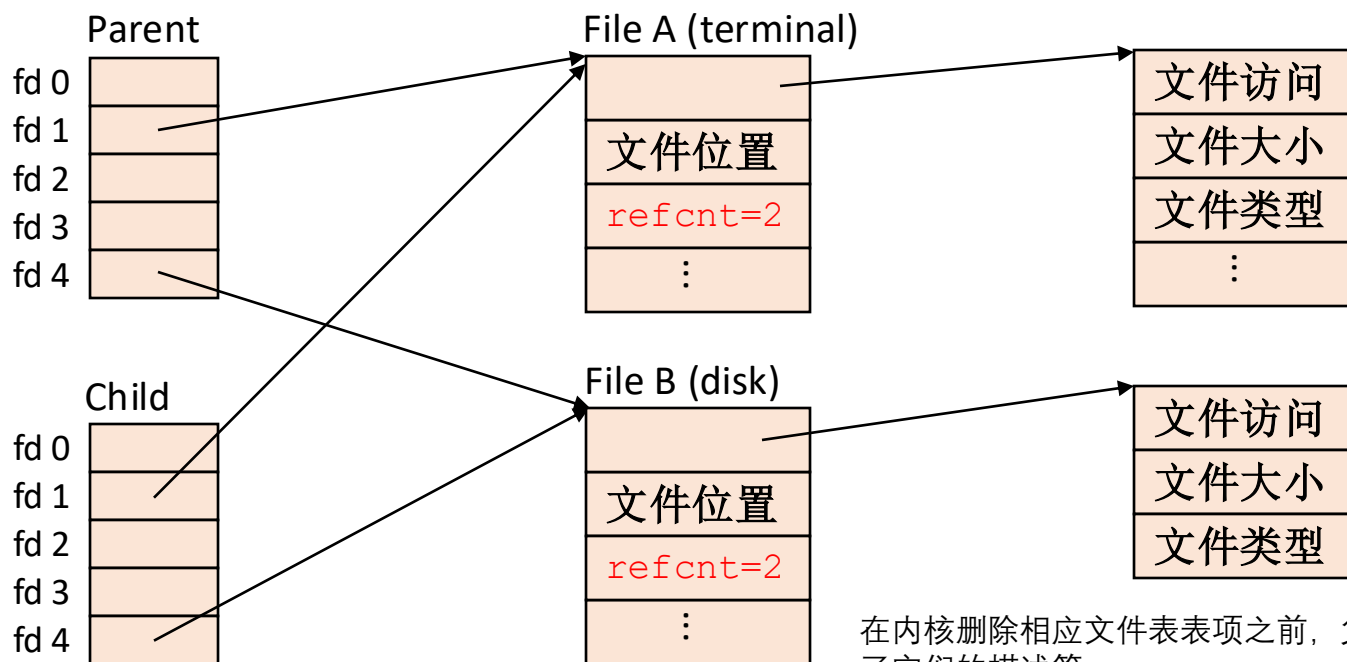
进程如何共享文件：fork（[阅读教材P515](#)）

- 子进程继承父进程的打开文件
- 调用fork **之后**:
 - 子进程的表与父进程的**表相同, 每一个 refcnt +1**

描述符表
[一个进程一个表]

打开文件表
[所有进程共享]

v-node 表
[所有进程共享]



在内核删除相应文件表表项之前，父子进程必须都关闭了它们的描述符

I/O Redirection

I/O重定向

- 问题: Unix内核如何实现 I/O 重定向?
 - 允许用户将磁盘文件和标准输入输出联系起来
 - **linux> ls > foo.txt**
- 回答: 通过调用 `dup2(oldfd, newfd)` 函数
 - 复制描述符表表项 **oldfd** 到描述符表表项 **newfd**, 覆盖描述符表表项 **newfd** 以前的内容。(记住: 左边描述符表项覆盖右边)

描述符表

调用 `dup2(4, 1)` 之前

fd 0	
fd 1	a
fd 2	
fd 3	
fd 4	b



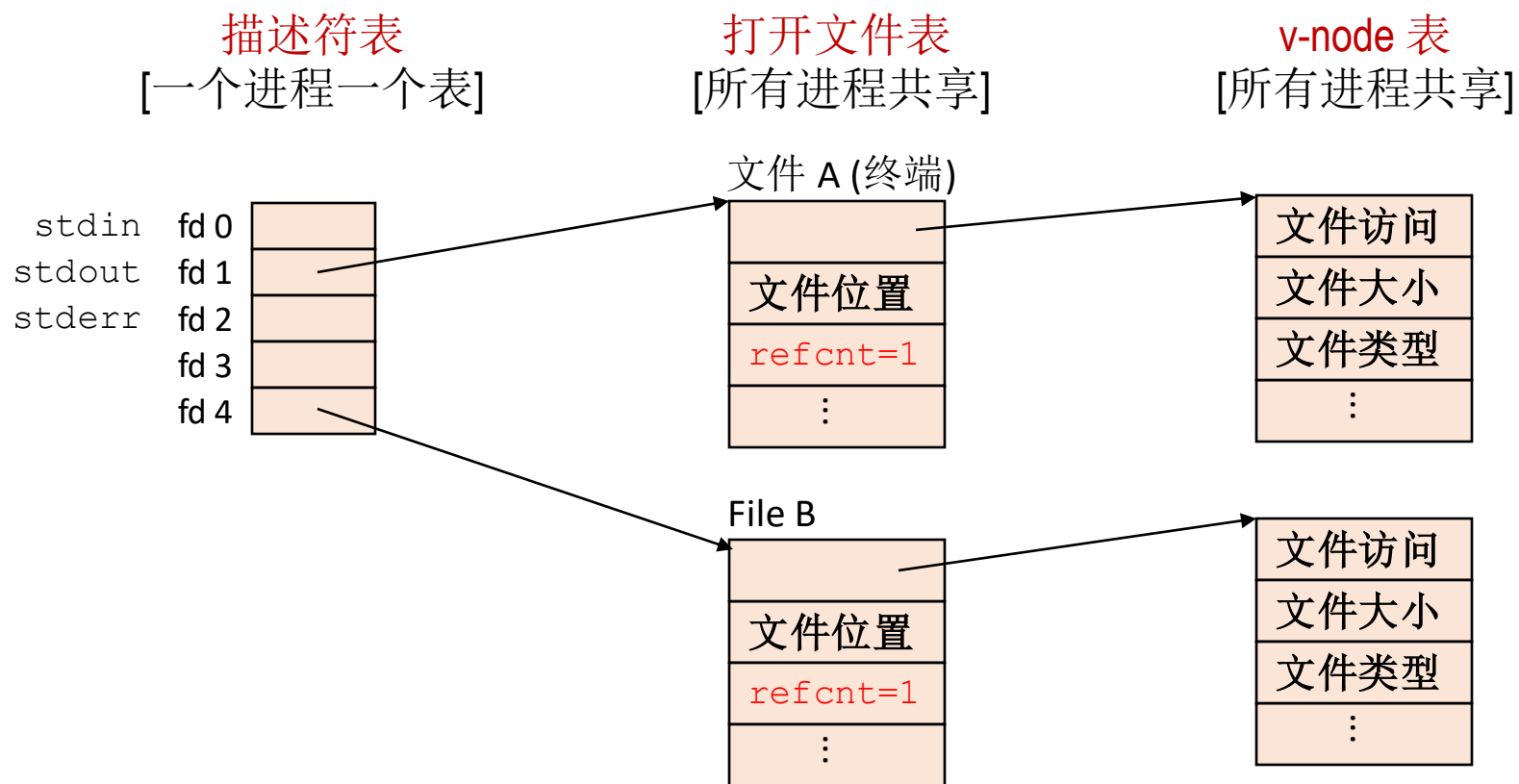
描述符表

调用 `dup2(4, 1)` 之后

fd 0	
fd 1	b
fd 2	
fd 3	
fd 4	b

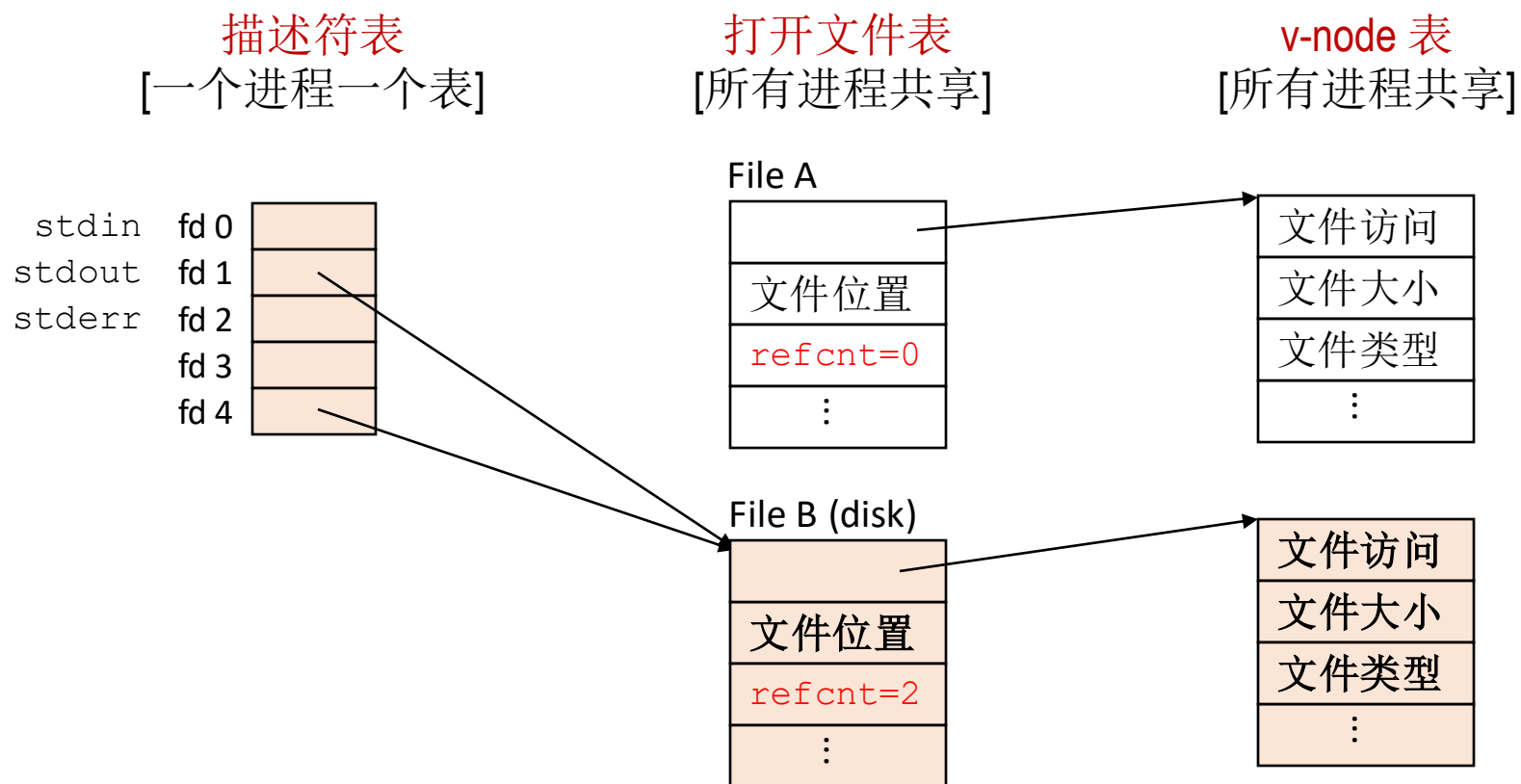
I/O Redirection Example

- 步骤 #1: 打开需重定位文件
 - 在调用dup2 (4, 1)之前



I/O Redirection Example

- 步骤 #2: 调用 `dup2 (4, 1)`
 - 使得 `fd=1 (stdout)` 指向 `fd=4` 所指向的磁盘文件



主要内容

- Unix I/O
- 用RIO包健壮地读写
- 读取文件元数据，共享和重定位
- 标准I/O
- 结束语

Standard I/O Functions 标准I/O 函数

- C语言定义了标准I/O库 (`libc.so`)，为程序员提供了Unix *标准I/O* 的较高级别的替代
 - 详见附录B中K&R的文章
- 标准 I/O 函数示例:
 - 打开和关闭文件 (**fopen** 和 **fclose**)
 - 读和写字节 (**fread** 和 **fwrite**)
 - 读和写字符串 (**fgets** 和 **fputs**)
 - 格式化的读和写 (**fscanf** and **fprintf**)

Standard I/O Streams 标准I/O流

- 标准 I/O库将一个打开的文件 模型化为 **流**
 - 对文件描述符和流缓冲区的抽象
- 每个C程序开始时都有三个打开的流(在stdio.h中定义)
 - **stdin** (standard input) 标准输入
 - **stdout** (standard output) 标准输出
 - **stderr** (standard error) 标准错误

```
#include <stdio.h>
extern FILE *stdin; /* standard input (descriptor 0) */
extern FILE *stdout; /* standard output (descriptor 1) */
extern FILE *stderr; /* standard error (descriptor 2) */

int main() {
    fprintf(stdout, "Hello, world\n");
}
```


Buffered I/O: Motivation 带缓冲I/O的动机

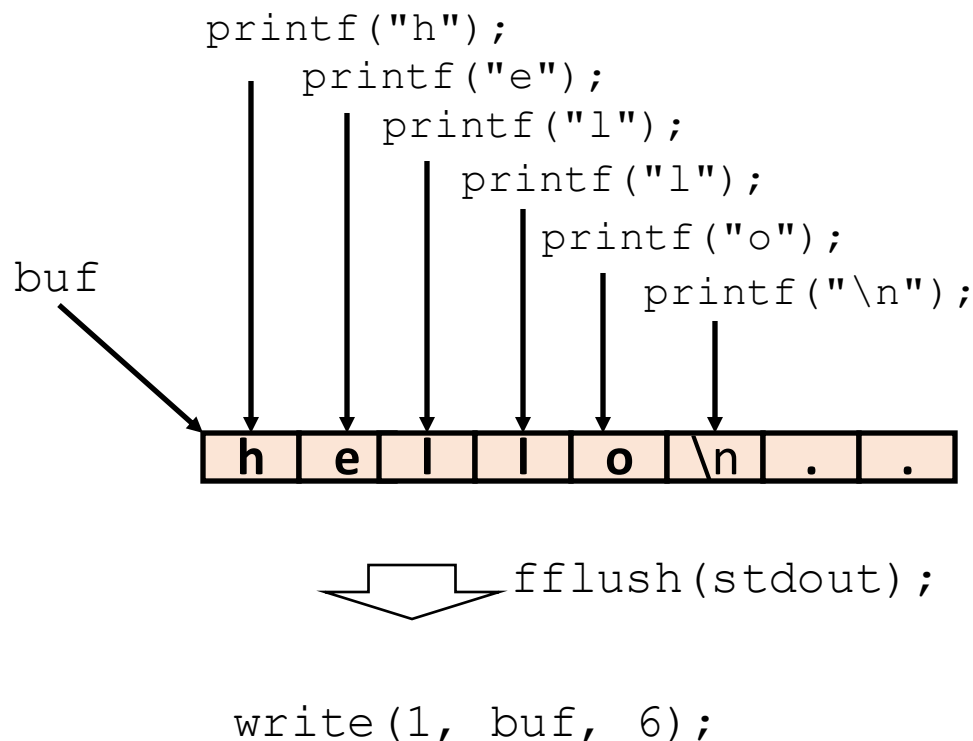
- 应用经常每次读/写一个字符
 - `getc`, `putc`, `ungetc`
 - `gets`, `fgets`
 - 每次读一行文本，到新行处停止
- 作为昂贵的 **Unix I/O** 调用来执行
 - 读和写需要调用 **Unix** 内核
 - > 10,000 时钟周期
- 解决: 带缓冲的读
 - 使用 **Unix** 读获取字符块
 - 用户输入函数每次从缓存取一个字节
 - 当缓存为空时重新填充

缓存



Buffering in Standard I/O 标准I/O的缓存

- 使用带缓冲的标准 I/O 函数



- 缓冲区刷新到输出 `fd`，当遇到“\n”，调用 `fflush` 或 `exit`，或从 `main` 返回。

标准I/O缓冲区的作用

- 通过Linux的 `strace` 程序观察这种缓冲作用:

```
#include <stdio.h>

int main()
{
    printf("h");
    printf("e");
    printf("l");
    printf("l");
    printf("o");
    printf("\n");
    fflush(stdout);
    exit(0);
}
```

```
linux> strace ./hello
execve("./hello", ["hello"], [/* ... */]).
...
write(1, "hello\n", 6)                = 6
...
exit_group(0)                         = ?
```

fflush与fsync

- 分别是标准I/O库函数和系统调用

功能	fflush	fsync
作用范围	用户态缓冲区到内核态缓冲区	内核态缓冲区到磁盘
是否持久化到磁盘	否	是
典型场景	实时输出或数据写入内核缓冲区	确保数据持久化到磁盘，防止数据丢失
调用位置	FILE *(标准库层面)	文件描述符(系统调用层面)

Linux I/O缓冲测试

- buffered.sh
 - 先查看内存占用
 - 使用dd向buffer_test文件写入数据（大小为16MB*256）
 - 最后再查看内存占用
- clear.sh
 - 清理内存buffer
- direct.sh
 - 先查看内存占用
 - 使用dd向buffer_test文件写入数据（大小为16MB*256），加入direct I/O 标记
 - 最后再查看内存占用

```
[zxy@dell-r760:~/IOTest$ cat buffered.sh
free -h
dd if=/dev/zero of=buffer_test bs=16M count=256
free -h
[zxy@dell-r760:~/IOTest$ cat clear.sh
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
[zxy@dell-r760:~/IOTest$ cat direct.sh
free -h
dd if=/dev/zero of=buffer_test bs=16M count=256 oflag=direct
free -h
```

Linux I/O缓冲测试

```
[zxy@dell-r760:~/IOTest]$ cat buffered.sh
free -h
dd if=/dev/zero of=buffer_test bs=16M count=256
free -h
```

```
[zxy@dell-r760:~/IOTest]$ cat clear.sh
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
```

```
[zxy@dell-r760:~/IOTest]$ cat direct.sh
free -h
dd if=/dev/zero of=buffer_test bs=16M count=256 oflag=direct
free -h
```

zouxiangyu — zxy@dell-r760: ~/IOTest — ssh • 10.249.188.30 — 209x26

zxy@dell-r760:~/IOTest\$

avg-cpu:	%user	%nice	%system	%iowait	%steal	%idle
	0.01	0.00	0.01	0.00	0.00	99.98

Device	r/s	rkB/s	rrqm/s	%rrqm	r_await	rareq-sz	w/s	wkB/s	wrqm/s	%wrqm	w_await	wareq-sz	d/s	dkB/s	drqm/s	%drqm	d_await	dareq-sz	f/s	f_await	aqu-sz	%util
loop0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop3	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
loop4	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
md1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
nvme0n1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
nvme1n1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sda	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sdb	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sdg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sdf	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
sdh	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Linux I/O缓冲

■ 如果不去手动的输入sync命令来真正的去写磁盘,

linux系统也会有两种写磁盘的时机:

- 1) kflush内核线程周期性的去写磁盘;
- 2) buffer已满不得不写。

Linux I/O缓冲

- 读操作也会利用缓冲区、后续访问相同文件会加速
- 使用特定命令可以清理缓存 echo 3
>/proc/sys/vm/drop_caches

```
[zxy@dell-r760:/mnt/ssd-r]$ free -h
              total        used        free      shared  buff/cache   available
Mem:           125Gi       1.8Gi       123Gi         4.0Mi        272Mi       122Gi
Swap:           8.0Gi        328Mi        7.7Gi

[zxy@dell-r760:/mnt/ssd-r]$ time md5sum 82.0.4066.0.tar
1b6259b5518342e2a1b30cea0ef867ee 82.0.4066.0.tar

real    0m6.006s
user    0m5.492s
sys     0m0.509s
[zxy@dell-r760:/mnt/ssd-r]$ free -h
              total        used        free      shared  buff/cache   available
Mem:           125Gi       1.8Gi       120Gi         4.0Mi         3.2Gi       122Gi
Swap:           8.0Gi        328Mi        7.7Gi

[zxy@dell-r760:/mnt/ssd-r]$ time md5sum 82.0.4066.0.tar
1b6259b5518342e2a1b30cea0ef867ee 82.0.4066.0.tar

real    0m5.761s
user    0m5.449s
sys     0m0.310s
[zxy@dell-r760:/mnt/ssd-r]$ sudo su
[root@dell-r760:/mnt/ssd-r# echo 3 > /proc/sys/vm/drop_caches
[root@dell-r760:/mnt/ssd-r# exit
exit
[zxy@dell-r760:/mnt/ssd-r]$ time md5sum 82.0.4066.0.tar
1b6259b5518342e2a1b30cea0ef867ee 82.0.4066.0.tar

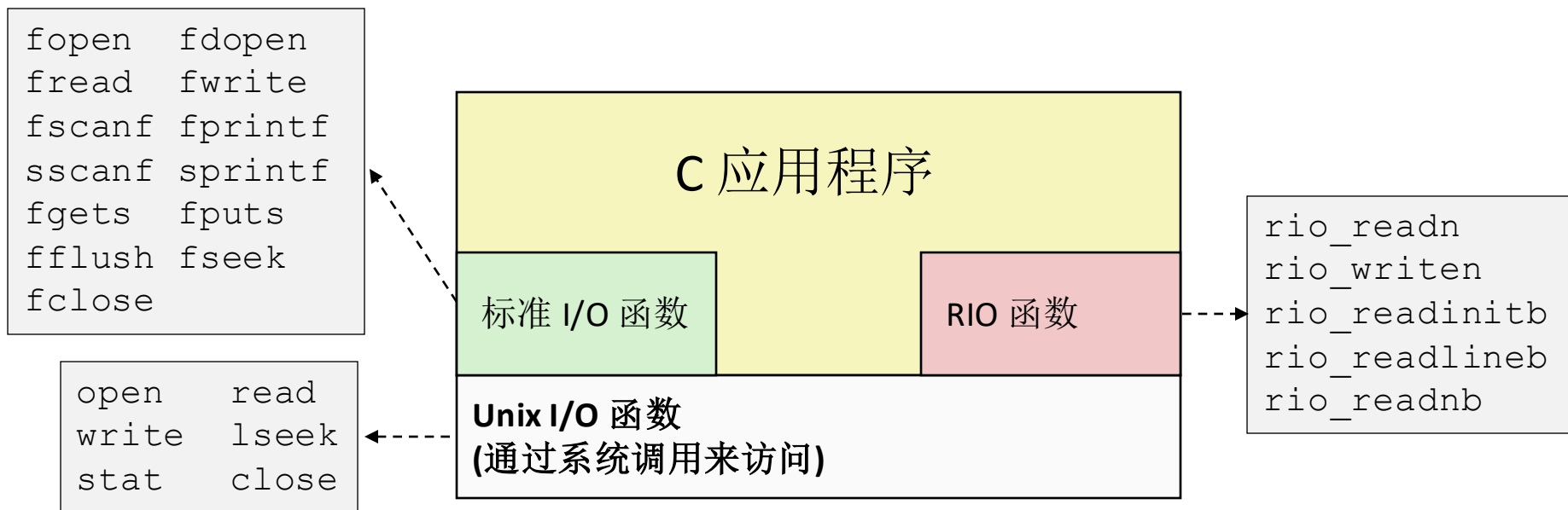
real    0m6.008s
user    0m5.498s
sys     0m0.503s
zxy@dell-r760:/mnt/ssd-r$
```


主要内容

- Unix I/O
- 用RIO包健壮地读写
- 读取文件元数据，共享和重定位
- 标准I/O
- 结束语

Unix I/O 、标准 I/O 和 RIO之间的关系

- 标准 I/O 和 RIO 是基于较低级的 Unix I/O 函数来实现的。



- 我该使用哪些I/O函数?

Unix I/O优点和缺点

• 优点

- Unix I/O 是最通用、开销最低的I/O方式
 - 所有其他 I/O都是使用Unix I/O 函数来实现的
- Unix I/O 提供访问文件元数据的函数
- Unix I/O 函数是异步信号安全的，可以在信号处理程序中安全地使用

• 缺点

- 处理不足值时容易出错
- 有效地读取文本行需要某种形式的缓冲, 容易出错
- 这两个问题都是由标准I/O和RIO包来解决

标准I/O的优点和缺点

- 优点:
 - 通过减少读和写系统调用的次数，从而减少间接带来的内存占用
 - 自动处理不足值
- 缺点:
 - 没有提供访问文件元数据的函数
 - 标准 I/O 函数不是异步信号安全的, 不适合用于信号处理
 - 标准 I/O 不适合网络套接字的输入输出操作
 - 对流的限制和对套接字的限制有时候会互相冲突，而又很少有文档描述这些现象(CS:APP3e, Sec 10.11)

I/O函数的选择（详见书P639）

- 一般规则：使用最高级别的I/O函数
 - 大多数 C 程序员在其整个职业生涯中只使用标准 I/O
 - 但是,他一定明白你所使用的函数!
- 什么时候使用标准 I/O
 - 当使用磁盘文件和终端文件时
- 什么时候使用 Unix I/O
 - 在信号处理程序中, 因为 Unix I/O 是异步信号安全的
 - 在极少数情况下, 当你需要绝对最高的性能时
- 什么时候使用 RIO
 - 当你准备读、写网络套接字时
 - 避免在套接字上使用标准I/O

习题

■ 1. 若将标准输出重定向到文本文件file.txt，错误的是：

- A. 需要先打开重定位的目标文件“file.txt”
- B. 设“file.txt”对应的fd为4，内核调用dup2(1, 4)函数实现描述符表项的复制
- C. 复制“file.txt”的打开文件表项，并修正fd为1的描述符
- D. 修改“file.txt”的打开文件表项的引用计数

■ B、C

解析：B选项正确的写法应该是dup2(4,1)，左边描述符表项覆盖右边。C选项错在最后“描述符”，因为“描述符”是不能随意修改的，而重定向的本质是描述符表项的复制。

*Hope you
enjoyed
the
CSAPP
course!*